

Linux Practical Practice Guide for DevOps

Linux Practical Practice Guide for DevOps

Hands-on exercises with explanations — to be done inside your `ubuntu-devops` VM

Goal: Don't just memorize commands — understand *why* each one matters in real DevOps work (deployments, monitoring, debugging production issues). Do every exercise yourself in the terminal; don't just read.

How to Use This Guide

- Log into your VM (`username` + `password` you set)
 - Work through sections in order — each builds on the last
 - Type every command yourself; don't copy-paste blindly — typing builds memory
 - Where it says “Expected output,” compare what you see — if different, that's a learning moment, not a failure
-

SESSION 1: Orientation & Navigation

Why this matters: Before touching servers, you need automatic, no-thinking confidence moving around the filesystem. In real incidents, you'll be SSH'd into a server at 2am with no GUI — speed here matters.

```
pwd
```

Explanation: Prints your current location. You're probably in `/home/yourusername`.

```
whoami
```

Explanation: Confirms which user you're logged in as. Important once you start using `sudo` and switching users.

```
ls
ls -l
ls -la
```

Explanation: - `ls` — basic list - `-l` — long format: shows permissions, owner, size, modified date - `-a` — shows hidden files (anything starting with `.`, like `.bashrc`)

Exercise: Run `ls -la` in your home directory. Identify: 1. Which files are hidden (start with `.`)? 2. What does the first character of each permission string mean (`-` vs `d`)?

```
cd /  
ls  
cd /etc  
ls  
cd ~
```

Explanation: `/` is the root of the entire filesystem (not to be confused with the root *user*). `/etc` holds system configuration files — you’ll live here often as a DevOps engineer. `~` always takes you home regardless of where you are.

Exercise: Navigate to `/var/log`, list what’s there, then return home using just one command.

SESSION 2: Creating, Editing, and Inspecting Files

Why this matters: You’ll constantly create config files, scripts, and check log files. This is muscle memory you need cold.

```
mkdir devops-practice  
cd devops-practice  
touch notes.txt  
ls -l
```

Explanation: Created a folder and an empty file inside it.

```
nano notes.txt
```

Explanation: Opens the `nano` text editor (beginner-friendly). Type some text, then: - `Ctrl+O` → save (it’ll ask to confirm filename, press Enter) - `Ctrl+X` → exit

```
cat notes.txt
```

Explanation: Prints file contents to screen.

Exercise: Create 3 files: `app.log`, `config.yaml`, `script.sh`. Add a different sentence to each using `nano`. Then `cat` all three to confirm.

```
cp notes.txt notes_backup.txt  
ls -l  
mv notes_backup.txt old_notes.txt  
ls -l  
rm old_notes.txt  
ls -l
```

Explanation: Copy → rename (`mv` renames if target is in the same folder) → delete. This is the exact pattern you’ll use when backing up configs before editing them in production — **always cp before you edit anything critical**.

SESSION 3: Permissions (Critical for DevOps)

Why this matters: Almost every “permission denied” error you’ll ever debug in your career traces back to understanding this section properly.

```
ls -l notes.txt
```

Expected output looks like:

```
-rw-r--r-- 1 yourusername yourusername 0 Jun 28 10:00 notes.txt
```

Explanation of the string `-rw-r--r--`: - First character: `-` = regular file (`d` would mean directory) - Next 3: `rw-` = owner can read+write, not execute - Next 3: `r--` = group can only read - Next 3: `r--` = everyone else can only read

```
chmod +x notes.txt
ls -l notes.txt
```

Explanation: Added execute permission for everyone. Watch how the string changes.

```
chmod 644 notes.txt
ls -l notes.txt
```

Explanation: Numeric method. `6` = `rw-` (owner), `4` = `r-` (group), `4` = `r-` (others). This is THE standard permission for normal files.

```
chmod 755 script.sh
ls -l script.sh
```

Explanation: `7` = `rwx` (owner can run it), `5` = `r-x` (group can run, not edit), `5` = `r-x` (others same). This is THE standard permission for scripts/executables.

Exercise: Write a one-line script in `script.sh`:

```
echo "Hello DevOps"
```

Save it. Try running it with `./script.sh` — if it fails with “permission denied,” fix the permissions yourself using what you just learned, then run it again until it prints “Hello DevOps.”

```
sudo chown yourusername:yourusername notes.txt
```

Explanation: Changes file owner and group. You won’t need this often on your own files, but you absolutely will when deploying apps as a different system user (e.g. `www-data` for web servers).

SESSION 4: Process Management

Why this matters: When an app crashes, hangs, or eats all the RAM on a server, this is how you investigate and fix it.

```
ps aux
```

Explanation: Lists every running process. Columns: USER, PID (Process ID), %CPU, %MEM, COMMAND.

```
ps aux | head -20
```

Explanation: Pipe (`|`) sends output of `ps aux` into `head`, which only shows first 20 lines. You’ll use pipes constantly.

```
top
```

Explanation: Live, refreshing view of processes by CPU/memory usage. Press `q` to quit.

Exercise: Start a background process that just sleeps:

```
sleep 300 &
```

Explanation: `&` runs it in the background and returns you to the prompt immediately. It'll print a PID.

```
jobs
```

Explanation: Shows background jobs in your current session.

```
ps aux | grep sleep
```

Explanation: Filters `ps aux` output to only show lines containing "sleep." This pattern (`ps aux | grep something`) is one of the most common DevOps debugging commands ever.

```
kill <PID>
```

Explanation: Replace `<PID>` with the actual process ID number you saw. This terminates it.

```
ps aux | grep sleep
```

Explanation: Confirm it's gone (you'll likely only see the `grep` command itself now, not the `sleep` process).

SESSION 5: Text Processing — `grep`, `awk`, `sed` (THE most-used DevOps skill)

Why this matters: Log analysis is daily work. A 100,000-line log file is unreadable by eye — these tools make it instant.

First, let's create a sample log file to practice on:

```
cd ~/devops-practice  
nano sample.log
```

Paste in this content (type it manually for practice):

```
2026-06-28 10:01:23 INFO User login successful user=alice  
2026-06-28 10:02:01 ERROR Database connection failed  
2026-06-28 10:02:45 INFO User login successful user=bob  
2026-06-28 10:03:12 ERROR Database connection failed  
2026-06-28 10:04:00 WARNING High memory usage detected  
2026-06-28 10:05:30 INFO User logout user=alice  
2026-06-28 10:06:15 ERROR Disk space low
```

Save and exit (`Ctrl+O` , `Enter`, `Ctrl+X`).

```
grep "ERROR" sample.log
```

Explanation: Shows only lines containing "ERROR." This is the #1 command for finding problems in logs.

```
grep -c "ERROR" sample.log
```

Explanation: `-c` counts matching lines instead of showing them. Quick way to see "how many errors happened?"

```
grep -i "warning" sample.log
```

Explanation: `-i` ignores case — matches "WARNING," "Warning," "warning," all the same.

```
grep -v "INFO" sample.log
```

Explanation: `-v` inverts the match — shows everything EXCEPT lines with “INFO.” Useful to filter out noise.

```
awk '{print $1, $2, $4}' sample.log
```

Explanation: `awk` treats each line as columns separated by spaces. `$1` = date, `$2` = time, `$4` = log level (ERROR/INFO/WARNING). This extracts just the columns you care about.

```
awk '/ERROR/ {print $1, $2}' sample.log
```

Explanation: Only prints date+time for lines matching “ERROR.” Combines filtering and extracting in one command.

```
sed 's/ERROR/CRITICAL/g' sample.log
```

Explanation: Find-and-replace. Replaces every “ERROR” with “CRITICAL” in the output (doesn’t change the actual file unless you add `-i`).

```
sed -i 's/ERROR/CRITICAL/g' sample.log  
cat sample.log
```

Explanation: `-i` edits the file in-place. Now check — the file itself has changed.

Exercise: Using `grep`, `awk`, and `sort | uniq -c`, find out how many times each user (`alice`, `bob`) appears in the log. (Hint: `grep "user=" sample.log | awk -F'user=' '{print $2}' | sort | uniq -c`)

SESSION 6: Real-time Log Monitoring

Why this matters: When you deploy something, you watch logs live to see if it’s working or breaking. This is the exact moment of truth in every deployment.

```
tail -f sample.log
```

Explanation: `-f` means “follow” — it keeps the terminal open and shows new lines as they’re added, live. Used constantly with real application logs.

Open a **second terminal** (or split your session) and run:

```
echo "2026-06-28 10:10:00 ERROR New critical issue" >> sample.log
```

Explanation: `>>` appends a line to the file. Watch your first terminal — the new line should appear instantly if `tail -f` is running.

Press `Ctrl+C` in the first terminal to stop following.

Real DevOps habit: `tail -f /var/log/syslog` or `journalctl -f` is something you’ll run dozens of times a week once you’re managing real servers.

SESSION 7: Package Management

Why this matters: Installing tools (nginx, docker, git, etc.) on servers is constant work.

```
sudo apt update
```

Explanation: Refreshes the list of available packages from the internet — always run this before installing anything.

```
sudo apt install tree -y
```

Explanation: Installs the `tree` tool (visual directory listing). `-y` auto-confirms the install prompt.

```
tree devops-practice
```

Explanation: See your practice folder as a visual tree.

```
sudo apt install htop -y  
htop
```

Explanation: `htop` is a nicer, color-coded version of `top`. Press `q` to quit.

```
sudo apt remove tree -y
```

Explanation: Uninstalls a package cleanly.

SESSION 8: systemd — Managing Services (Core DevOps Skill)

Why this matters: Every web server, database, and background service on a Linux server is managed by systemd. This is non-negotiable knowledge for DevOps.

```
systemctl status ssh
```

Explanation: Shows whether the SSH service is running, since when, and recent log lines. Should show “active (running)” since you enabled SSH during install.

```
sudo systemctl stop ssh  
systemctl status ssh
```

Explanation: Stopped the service — status should now show “inactive (dead).”

```
sudo systemctl start ssh  
systemctl status ssh
```

Explanation: Started it again — back to “active (running).”

```
sudo systemctl restart ssh
```

Explanation: Stop + start in one command — used after changing a service’s config file.

```
sudo systemctl enable ssh
```

Explanation: Makes sure SSH starts automatically every time the server boots (probably already enabled).

```
journalctl -u ssh -n 20
```

Explanation: Shows the last 20 log lines specifically for the SSH service. `-u` = unit (service name), `-n` = number of lines.

Exercise: Create a fake custom service (advanced but very instructive):

```
sudo nano /etc/systemd/system/myhello.service
```

Paste this in:

```
[Unit]
Description=My Hello Service

[Service]
ExecStart=/bin/bash -c 'while true; do echo "Hello from systemd"; sleep 5; done'

[Install]
WantedBy=multi-user.target
```

Save and exit. Then:

```
sudo systemctl daemon-reload
sudo systemctl start myhello
systemctl status myhello
journalctl -u myhello -f
```

Explanation: You just created, started, and watched the logs of your own systemd-managed service — exactly how real applications run in production. Press `Ctrl+C` to stop following logs, then:

```
sudo systemctl stop myhello
sudo systemctl disable myhello
```

SESSION 9: Networking Basics

Why this matters: You'll constantly check if a server can reach the internet, if a port is open, or if a service is listening correctly.

```
ip a
```

Explanation: Shows network interfaces and their IP addresses. Look for `enp0s3` or similar — that's your VM's IP on the NAT network.

```
ping -c 4 google.com
```

Explanation: Sends 4 ping packets to check internet connectivity. `-c 4` limits it to 4 (otherwise it runs forever).

```
ss -tulnp
```

Explanation: Shows all listening ports and which process owns them. You should see port 22 (SSH) listed.

```
curl -I https://www.google.com
```

Explanation: Sends an HTTP request and shows just the response headers (`-I`). Common way to test if a web service is responding.

SESSION 10: SSH — Connecting From Your Real Computer (Big Milestone)

Why this matters: This is the moment your practice VM starts feeling like a real cloud server — connecting remotely instead of using the VirtualBox window.

First, find your VM's IP:

```
ip a
```

Look for something like `10.0.2.15` under `enp0s3` (NAT default range).

From your Windows machine (not inside the VM), open **Command Prompt** or **PowerShell** and run:

```
ssh yourusername@10.0.2.15
```

Explanation: Connects to your VM over SSH using its IP. You'll be asked to trust the host fingerprint the first time — type `yes`. Then enter your password.

Note: With VirtualBox's default NAT networking, this might not work directly from your host machine — NAT isolates the VM. If it fails, that's expected and we can fix it later by switching the VM's network adapter to "Bridged" mode. For now, it's fine to practice everything inside the VirtualBox window directly.

SESSION 11: Shell Scripting — Putting It All Together

Why this matters: Real DevOps work is automating repetitive tasks. A script that checks disk space and warns you is a perfect beginner project.

```
nano diskcheck.sh
```

Paste this:

```
#!/bin/bash
echo "Checking disk usage..."
USAGE=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')

if [ "$USAGE" -gt 80 ]; then
    echo "WARNING: Disk usage is at ${USAGE}% - action needed!"
else
    echo "Disk usage OK: ${USAGE}%"
fi
```

Save and exit.

```
chmod +x diskcheck.sh
./diskcheck.sh
```

Explanation walkthrough: - `df /` shows disk usage for the root filesystem - `tail -1` grabs just the data row (skips header) - `awk '{print $5}'` grabs the percentage column - `sed 's/%//'` strips the % symbol so we have a plain number - `if [ "$USAGE" -gt 80 ]` checks if usage is over 80% - This is a real, working monitoring script — the same pattern used in production alerting

Exercise: Modify the script to also check memory usage using `free` and add a similar warning if memory usage is too high.

SESSION 12: Cron — Scheduling Your Script

Why this matters: Scripts like the one above are useless unless they run automatically and regularly.

```
crontab -e
```


Explanation: Opens your personal cron table for editing (choose `nano` if asked which editor).

Add this line at the bottom:

```
*/5 * * * * /home/yourusername/devops-practice/diskcheck.sh >> /home/yourusername/devops-practice/diskcheck-output.log 2>&1
```

Explanation: Runs your script every 5 minutes, and saves both normal output and errors (`2>&1`) into a log file.

Save and exit. Wait 5 minutes, then check:

```
cat diskcheck-output.log
```

Explanation: You should see your script's output logged automatically — this is real automation, exactly how production health checks work.

Remove the cron job once you've confirmed it worked (no need to keep it running forever during practice):

```
crontab -e
```

Delete the line you added, save, exit.

Suggested Practice Schedule

Day	Sessions	Focus
1	1-3	Navigation, files, permissions
2	4-5	Processes, text processing
3	6-7	Log monitoring, package management
4	8	systemd (spend extra time here — it's critical)
5	9-10	Networking, SSH
6	11-12	Scripting, cron
7	Review	Redo every exercise without looking at this guide

After This Guide: Next Steps

Once you're comfortable with everything above: 1. Deploy one of your actual GitHub projects (e.g. AskBot) onto this VM 2. Set it up as a systemd service so it auto-restarts if it crashes 3. Write a deployment script that pulls the latest code from GitHub and restarts the service 4. This combination — real app + systemd + scripting — is genuinely a DevOps portfolio piece you can talk about in interviews

You now have a real, working Linux server. Everything from here on is just doing real things on it.